SemaAI Backend Workflow

Overview

SemaAI is a Swahili language learning platform that provides conversational practice with AI tutors. The backend implements a deterministic pipeline that processes user utterances through several stages: speech-to-text, morphological analysis, memory-augmented generation, and text-to-speech.

Workflow Diagram

Due to limitations in this environment, the workflow diagram is described in the detailed workflow section below.

Detailed Workflow

1. Input Handling

- User provides audio recording or text input
- Audio is processed through Speech-to-Text (STT) using Faster-Whisper (local) or cloud Whisper
- Text input bypasses STT entirely

2. Morphological Analysis (Stemmer)

- Every utterance undergoes deterministic Swahili morphological analysis
- The stemmer identifies verb tense markers and checks for conflicts with time cue words (jana=past, kesho=future)
- Only errors trigger diagnostic injection to save prompt tokens
- Correct sentences pay no extra token cost for diagnostics

3. Memory-Augmented Generation

- Uses LlamaIndex ChatMemoryBuffer with periodic summarization
- ErrorCache tracks repeated mistakes to provide one-line reminders instead of full re-teaching
- Memory is rebuilt from persisted session history on each turn
- Summarization occurs every N turns to control context window costs

4. LLM Tutor Generation

- Structured prompt includes:
  - System template defining AI character role (vendor, nurse, etc.)
  - Conversation history from memory
  - Structured diagnostic block (only when stemmer detects error)
  - Instructions for JSON output format
- LLM (Mistral/OpenAI/Gemini) generates:
  - Swahili reply in character
  - English translation
  - Structured feedback (score, corrections, good points, suggestion)

5. Output Generation

- AI response stored in conversation memory
- Text-to-Speech synthesis using edge-tts (Microsoft neural Swahili voices)
- Audio cached per (text, voice, rate) to avoid redundant synthesis
- Feedback metrics displayed to user (score, comprehension, grammar, enunciation)

6. Persistence

- Session state saved after each turn
- Includes updated conversation history and turn count

- Sessions auto-complete after configured maximum turns
- In-memory storage (swapable for Postgres/Redis in production)

Key Technical Features

Deterministic Stemmer

- Rule-based Swahili morphological segmenter
- Identifies: [INF ku-] [NEG] [SC] [TAM] [OBJ] STEM + FINAL_VOWEL
- Surface-level heuristics with confidence scoring
- Focuses on patterns most relevant for language learners

Cost Optimization

- Diagnostic injection only on errors (saves prompt tokens)
- ErrorCache prevents repetitive feedback
- Memory summarization reduces long-session costs
- JSON-only LLM output reduces parsing overhead

Reliability Features

- Provider fallbacks (Mistral → OpenAI → Gemini)
- Tenacity-based retry with exponential backoff
- Graceful degradation when TTS/STT unavailable
- Structured output validation with fallback parsing

Language Learning Focus

- Tense/marker error detection with time cues
- Culturally appropriate AI characters (vendors, nurses, etc.)
- Immediate feedback with concrete corrections
- Vocabulary breakdown showing morphological analysis
- Progressive difficulty scenarios (beginner → advanced)

Data Flow

1. Audio/Text Input → STT (if audio) → Transcript
2. Transcript → Stemmer Analysis → StemDiagnostic (if error)
3. Transcript + Diagnostic + Memory Context → LLM Prompt
4. LLM → Structured JSON Response (ai_text, ai_translation, feedback)
5. Response → TTS Synthesis (optional) + Memory Update
6. Output: Audio + Text Feedback + Metrics → User
7. Session State → Persistent Storage

Configuration Points

- STT Engine: local_faster_whisper vs cloud_whisper
- LLM Provider: mistral vs openai vs gemini (with fallbacks)
- TTS Engine: edge-tts vs none
- Memory Settings: token_limit, summarize_every, keep_recent
- Stemmer: enabled/disabled diagnostics
- Session Limits: max_turns per session

Error Handling

- STT failures fall back to text input
- LLM provider failures trigger automatic fallback
- TTS failures return empty audio (degrade gracefully)

- JSON parsing failures use coercion to extract ai_text
- All errors surface as user-friendly messages in UI